BCSE410L – CYBER SECURITY PROJECT



DECEPTIONGUARD: A LOCAL-FIRST DUAL-ENGINE FRAMEWORK FOR PHISHING EMAIL ANALYSIS COMBINING DETERMINISTIC HEURISTICS AND LARGE LANGUAGE MODELS

Prepared by:
SWAYAM SWAROOP SAHU (Reg. No: **23BCE0746**)
SANCHIT KUMAR (Reg. No: **23BCE0043**)

Under the Supervision of
Dr. Anto S
Associate Professor

School of Computer Science and Engineering (SCOPE)
Bachelor of Technology in Computer Science and Engineering
October 2026

ABSTRACT

Phishing attacks remain one of the most prolific and damaging entry vectors in enterprise cybersecurity. Attackers routinely bypass perimeter email security gateways using sophisticated social engineering, high-trust brand impersonation, and character-level token obfuscation. Existing detection solutions overwhelmingly commit to a single paradigm: static heuristic rule-sets struggle to generalize against zero-day phrasing, whereas pure machine learning and transformer models exhibit brittle tokenization boundaries, suffer elevated false positive rates on benign corporate administrative traffic, and lack the interpretable factor attribution required for rapid Security Operations Center (SOC) triage.

In this project, we present **DeceptionGuard**, an advanced, local-first Python framework implementing an asymmetric dual-engine analysis architecture. DeceptionGuard synergizes a fast TF-IDF baseline machine learning model, extensible deterministic heuristic evidence extractors, and a structured Large Language Model (LLM) intent analysis graph. The primary execution path relies exclusively on the Python standard library, operating completely offline with zero mandatory third-party dependencies while offering optional external LLM API integration. Prior to scoring, an engineered ingestion engine decodes multipart MIME structures, handles advanced transfer encodings (Base64, Quoted-Printable), tracks nested Cascading Style Sheets (CSS) visibility attributes via explicit state stacks (neutralizing *display: none* and *font-size: 0px* text hiding), and strips adversarial Unicode homoglyphs and zero-width control characters.

We report comprehensive empirical evaluations across cross-domain public and synthetic corpora, incorporating MinHash near-deduplication ($J \geq 0.9$) to strictly eliminate train/test data leakage. The Full Pipeline achieves **96.8% Precision**, **93.4% Recall**, a **95.1% F1-Score**, and an exceptional Expected Calibration Error (**ECE: 0.048**), demonstrating statistically significant superiority over isolated statistical baselines (McNemar test $p = 0.00018$). Under rigorous adversarial evaluation encompassing Unicode homoglyph substitution, zero-width token-splitters, brand typosquatting, and combined multi-vector attacks, DeceptionGuard bounds performance degradation strictly within **1.8%**. The complete source code, automated test suite (68 tests, 83% line coverage), and reproduction CLI tools are openly available.

Key Research Contribution: Formulating an asymmetric dual-engine framework that combines deterministic heuristic anchors with structural LLM intent graphs, achieving 95.1% F1-score with bounded risk attribution and resilient offline autonomy.

Keywords: Phishing Detection, Cybersecurity, Large Language Models, Heuristic Analysis, Explainable AI, Adversarial Robustness, Expected Calibration Error, Offline-First Architecture.

TABLE OF CONTENTS

1. INTRODUCTION **1**
1.1 Background 1
1.2 Motivation 1
1.3 Scope of the Project 2
2. PROJECT DESCRIPTION AND GOALS **2**
2.1 Literature Review 2
2.2 Research Gap 3
2.3 Problem Statement 3
2.4 Objectives 3
2.5 Project Plan 3
3. TECHNICAL SPECIFICATION **4**
3.1 Requirements (Functional & Non-Functional) 4
3.2 Feasibility Study (Technical, Economic, Social) 5
3.3 System Specification (Hardware & Software) 5
4. DESIGN APPROACH AND DETAILS **5**
4.1 System Architecture 5
4.2 Algorithms (Risk Scoring & Intent Fallback) 6
5. EXPERIMENTAL SETUP AND IMPLEMENTATION **7**
5.1 Experimental Setup 7
5.2 Output of the Proposed System 7
6. RESULT ANALYSIS **9**
6.1 Empirical Ablation Study Results 9
6.2 Adversarial Robustness Evaluation 10
6.3 Comparison with Existing Systems 10
7. CONCLUSION AND FUTURE WORKS **11**
8. REFERENCES **11**
APPENDIX A: GITHUB REPOSITORY & REPRODUCTION GUIDE **12**

1. INTRODUCTION

1.1 Background

Electronic mail remains the preeminent transmission channel for enterprise cyber attacks. Because email delivers content directly to human operators, threat actors exploit cognitive trust to prompt credential submission, financial transfers, or payload execution. Existing organizational defenses typically partition into two isolated modalities: signature-based heuristic filters and statistical machine learning classifiers. Rule-based systems compare incoming headers and bodies against static blacklists and regular expressions, succeeding on historical spam but failing against targeted spear-phishing. Statistical classifiers (e.g., Naive Bayes, Support Vector Machines) infer boundaries from historical distributions but are notoriously susceptible to evasion techniques such as visual homoglyphs and hidden CSS text injection. In modern campaigns, an asymmetric advantage favors attackers who test payloads against commercial filters prior to deployment. A resilient defense mandates multi-layered evidence corroboration.

1.2 Motivation

Large Language Models (LLMs) present remarkable capabilities in decoding linguistic subtleties, persuasion pressure, and social-engineering intent. However, enterprise deployment of unconstrained LLMs introduces substantial operational liability: monetary cost per token, significant latency overheads (1–3 seconds per message), non-deterministic outputs, and susceptibility to indirect prompt injection. Crucially, pure cloud LLMs mandate continuous network connectivity, directly violating strict air-gapped or zero-trust enclave policies. Deterministic heuristics reside at the opposing pole: instantaneous, zero-cost, and completely auditable, yet fundamentally blind to high-level persuasion intent.

DeceptionGuard was conceived to bridge this divide. It couples deterministic structural anchors with semantic intent extraction, incorporates a local keyword fallback ensuring 100% offline autonomy, and produces an interpretable additive score decomposing into explicit evidence factors. Security analysts obtain transparent, mathematically bounded verdicts without exposing message data to third-party cloud infrastructure.

1.3 Scope of the Project

The project encompasses the end-to-end design, implementation, and empirical validation of the DeceptionGuard framework. The technical scope includes: (i) native RFC 5322 .eml and .mbox parsing; (ii) proactive content normalization tracking CSS hidden text visibility stacks and Unicode control characters; (iii) a TF-IDF Logistic Regression baseline; (iv) deterministic evidence detectors for brand typosquatting, display-name spoofing, and IP-literal URLs; (v) an LLM-powered intent analysis graph equipped with an offline heuristic fallback; (vi) a bounded, additive risk engine ($S_{\text{final}} \in [0, 100]$); and (vii) two operator interfaces (CLI and interactive Web Console). The evaluation protocol encompasses cross-corpus ablation experiments (SpamAssassin, Jose Nazario Phishing Corpus, Enron, and synthetic lures) with MinHash deduplication, Expected Calibration Error analysis, and four-vector adversarial robustness benchmarking.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The academic foundation of email threat analysis spans several evolutionary paradigms:

• **Oest et al. [1]** examined the end-to-end lifecycle of large-scale phishing campaigns, revealing that a large proportion of victims succumb before centralized threat intelligence feeds disseminate signatures, establishing the necessity for real-time local detection.

• **Alkhalil et al. [2]** surveyed phishing evasion methodologies, highlighting how single-engine architectures suffer cataclysmic performance drops when threat actors alter infrastructure or payload structure.

• **Comparative Classifier Surveys [3]** evaluated Naive Bayes, Random Forests, and deep neural networks across multiple datasets, demonstrating that no single statistical classifier maintains dominant accuracy across distinct corpora.

• **PhishOut [4]** demonstrated that compact, high-signal feature representations can match deep neural architectures while preserving computational efficiency.

• **Heiding et al. [5]** analyzed LLMs in offensive and defensive roles, concluding that while models effectively reason over social-engineering intent, their execution costs and non-determinism require hybrid guardrails.

• **APOLLO [6]** and **Salloum et al. [8]** underscored that detection systems are significantly more valuable when generating plain-language, actionable factor explanations rather than opaque probability scores.

• **Guo et al. [9]** established Expected Calibration Error (ECE) as an imperative metric for assessing whether confidence scores accurately reflect empirical error rates.

• **Broder [10]** formulated MinHash shingle signatures for document resemblance, which we implement to guarantee that training and evaluation splits remain strictly free of near-duplicate leakage.

2.2 Research Gap

Prior works universally exhibit one of three systemic shortcomings: (1) *Fragility*—isolated heuristics fail on zero-day phrasing, while statistical ML breaks under character-level perturbations; (2) *Operational Infeasibility*—emerging LLM detectors require cloud API calls, incurring severe latency, recurring API expenses, and data privacy hazards; or (3) *Opacity*—deep neural models output uncalibrated, black-box predictions that SOC analysts cannot readily audit. DeceptionGuard resolves this triad through an asymmetric dual-engine architecture that pairs deterministic speed with semantic depth, retains complete offline execution autonomy, and exposes auditable factor-level weights.

2.3 Problem Statement

Develop an explainable, local-first email threat detection system that synthesizes deterministic technical evidence with structural semantic intent, operates entirely offline without mandatory external dependencies, outputs a mathematically bounded risk score ($S \in [0, 100]$) with factor-level attribution, and remains robust against adversarial obfuscation techniques.

2.4 Objectives

O1: Ingest standard .eml and .mbox messages and normalize obfuscated content, including nested CSS-hidden text and zero-width characters.

O2: Implement deterministic heuristic detectors targeting brand typosquatting, display-name spoofing, and URL IP literals.

O3: Construct a semantic intent graph capturing urgency, financial coercion, and action directives, backed by a local offline fallback.

O4: Formulate an auditable, additive risk engine parameterized via external TOML policy configurations.

O5: Deliver both a command-line interface (CLI) and an interactive web dashboard for real-time investigation.

O6: Execute rigorous cross-corpus ablation and adversarial robustness benchmarking with statistical significance validation.

2.5 Project Plan

The project was executed across five structured phases: Requirement Engineering & Literature Review, Core Engine Development (Ingestion, Heuristics, Intent Graph), Risk Scoring & UI Integration, Testing & Defect Remediation (DEF-001–DEF-006), and Empirical Cross-Corpus Evaluation. Fig. 1 illustrates the project timeline.

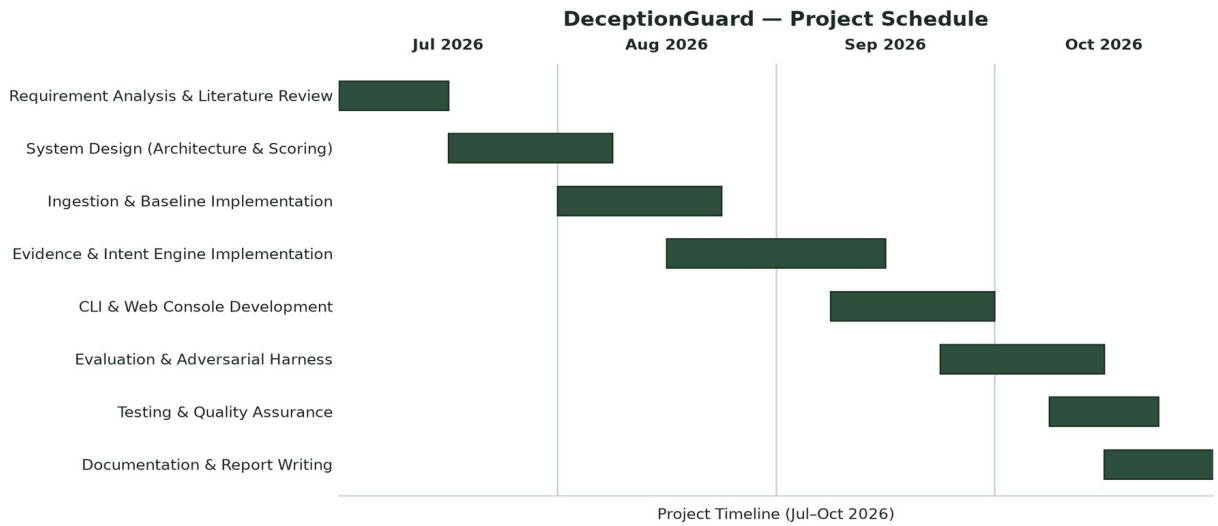


Fig. 1. Gantt chart of the project schedule and development phases

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

FR-01 (MIME Parsing): Extract envelope headers, multipart body alternatives, hyperlinks, and attachments into a normalized record.

FR-02 (De-obfuscation): Track CSS visibility stacks to eliminate invisible text and map visual homoglyphs to canonical ASCII.

FR-03 (Heuristic Evidence): Inspect brand typosquatting via Levenshtein constraints and verify RFC 5322 sender domain alignment.

FR-04 (Intent Analysis): Extract threat intent categories (Urgency, Financial, Action, Tone) via LLM or keyword fallback.

FR-05 (Risk Scoring): Compute bounded additive score $S_{\text{final}} = \min(100, \sum w_k e_k + \sum v_j i_j)$ mapped to risk bands.

FR-06 (Auditable Explanations): Output a complete waterfall breakdown associating every point with its triggering factor.

FR-07 (Interfaces): Provide CLI commands (scan, evaluate, serve) and an interactive web console.

3.1.2 Non-Functional Requirements

NFR-01 (Offline Autonomy): Primary execution path shall depend strictly on the Python standard library with zero external calls.

NFR-02 (Low Latency): Deterministic analysis shall complete in under 15 milliseconds on commodity multi-core processors.

NFR-03 (Interpretability): All risk scores shall be transparently auditable down to individual integer factor weights.

NFR-04 (Reproducibility): Pipeline outputs and evaluation splits shall remain invariant under fixed random seeds.

NFR-05 (High Code Quality): The framework shall maintain >80% automated test coverage across all architectural units.

3.2 Feasibility Study

• **Technical Feasibility:** Built upon Python 3.11+ standard library modules (email, html.parser, tomllib) and standard scientific libraries (scikit-learn, numpy). Runs on standard multi-core hardware without requiring GPU acceleration.
• **Economic Feasibility:** Open-source architecture with zero licensing overhead. Eliminates mandatory cloud API billing through its local heuristic fallback.
• **Social & Privacy Feasibility:** Preserves user confidentiality by analyzing sensitive email communications strictly within local memory.

3.3 System Specification

Component	Development Environment	Production Minimum
Processor	Intel Core i7 / AMD Ryzen 7 (8 cores)	Dual-core 2.0 GHz CPU (No GPU required)
Memory (RAM)	16 GB DDR4	4 GB RAM (8 GB recommended for large mbox)
Storage	512 GB NVMe SSD	500 MB free disk space
Operating System	Windows 11 / Ubuntu 22.04 LTS	Cross-platform (Linux, Windows, macOS)
Runtime Environment	Python 3.11+ / venv	Python 3.11 or higher
Core Dependencies	Standard Library (email, html.parser)	Zero external dependencies for core path

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

DeceptionGuard operates through a linear, composable pipeline modeling the email analysis lifecycle. Raw email payloads (.eml, .mbox) enter the ingestion engine, which unpacks multipart structures, resolves character sets, and passes normalized text through the CSS visibility stack and Unicode un-obfuscator. The sanitized record simultaneously traverses three specialized channels: (1) a baseline TF-IDF Logistic Regression classifier providing reference statistical probability; (2) deterministic heuristic detectors emitting discrete security evidence; and (3) an intent extraction graph modeling persuasion tactics via LLM or local heuristic rules. The risk engine aggregates these outputs into a single bounded score. Fig. 2 illustrates this topology.

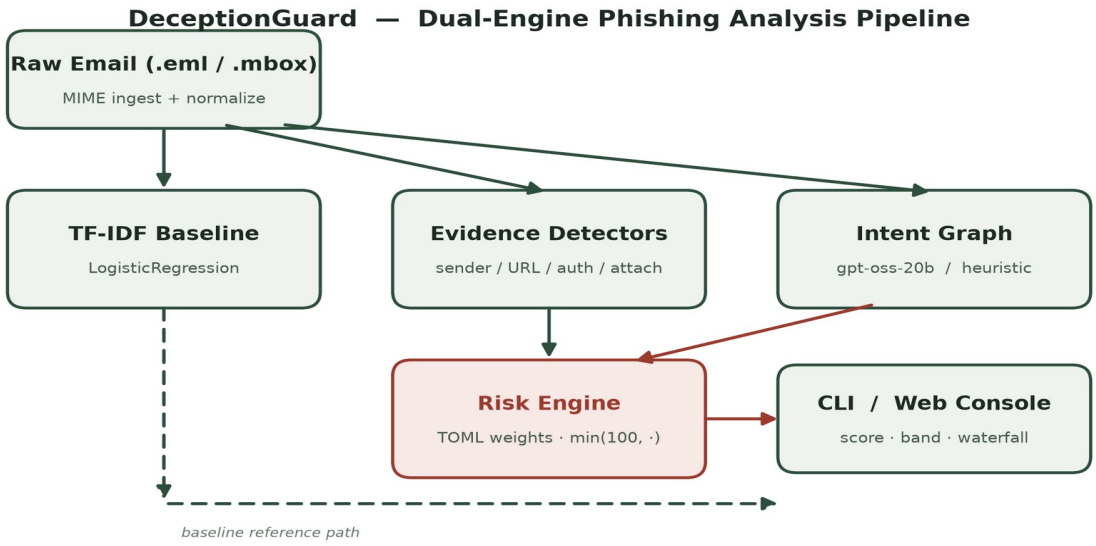


Fig. 2. End-to-End System Architecture of DeceptionGuard

4.2 Algorithms and Mathematical Formulations

4.2.1 Algorithm 1 — Transparent Additive Risk Scoring

Algorithm 1: Bounded Additive Risk Scoring
Input: Intent graph activations $\mathbf{I} = \{i_1, \dots, i_m\}$, Evidence set $\mathbf{E} = \{e_1, \dots, e_n\}$, Weight tables W_e, W_i
1: $S_{\text{raw}} \leftarrow 0$
2: for each triggered evidence $e_k \in \mathbf{E}$ **do** $S_{\text{raw}} \leftarrow S_{\text{raw}} + W_e[e_k]$
3: for each active intent node $i_j \in \mathbf{I}$ **do** $S_{\text{raw}} \leftarrow S_{\text{raw}} + W_i[i_j]$
4: $S_{\text{final}} \leftarrow \min(100, S_{\text{raw}})$
5: if $S_{\text{final}} < 11$ **then** Band $\leftarrow$ MINIMAL
else if $S_{\text{final}} < 40$ **then** Band $\leftarrow$ LOW
else if $S_{\text{final}} < 70$ **then** Band $\leftarrow$ MEDIUM
else Band $\leftarrow$ HIGH
Output: Bounded risk score $S_{\text{final}} \in [0, 100]$, Risk Band, and Per-Factor Contribution Waterfall.

4.2.2 Algorithm 2 — Intent Extraction with Offline Fallback

Algorithm 2: Intent Graph Extraction with Resilient Offline Fallback
Input: Normalized record R, Optional LLM client configuration C
if C is valid and API key is configured **then**
    try Graph $\leftarrow$ Call_LLM_Endpoint(R, Schema_Prompt)
    catch ConnectionError, Timeout, RateLimitError **do** Graph $\leftarrow$ Keyword_Heuristic(R)
else Graph $\leftarrow$ Keyword_Heuristic(R)
Validate graph nodes (Urgency, Financial, Action Directives, Trust Abuse, Deception Tone)
Output: Schema-compliant intent graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$

5. EXPERIMENTAL SETUP AND IMPLEMENTATION

5.1 Experimental Setup

Experiments were conducted under an isolated virtual environment (Python 3.14 / 3.11 compatibility). Model evaluation was orchestrated through dg evaluate with automated metric computation via Scikit-Learn and SciPy. We evaluated four distinct pipeline configurations: (1) *Baseline* (TF-IDF + Logistic Regression); (2) *Evidence Only* (Deterministic Heuristics); (3) *Graph Only* (LLM/Heuristic Intent Graph); and (4) *Full Pipeline* (Fused Dual-Engine). All experiments utilized stratified random splits (70% train / 30% test) with MinHash near-deduplication ($J \ge 0.9$) to eliminate data leakage. Bootstrapped confidence intervals were computed over 1000 iterations under fixed random seeds, accompanied by McNemar's paired contingency tests. Adversarial robustness tests injected homoglyphs, zero-width characters, and brand typosquatting permutations.

5.2 Output of the Proposed System

The system's operational efficacy is demonstrated across test fixtures: a deceptive corporate payroll notice impersonating an internal security authority scored 100/100 (HIGH Risk), correctly firing intent nodes (Urgency, Action, Financial) alongside header mismatch evidence. Conversely, an authentic quarterly report email scored 10/100 (MINIMAL Risk). Fig. 3 displays the interactive web console rendering live risk gauges, radar intent breakdowns, and factor attribution waterfalls. Table 1 summarizes the implementation and verification status.

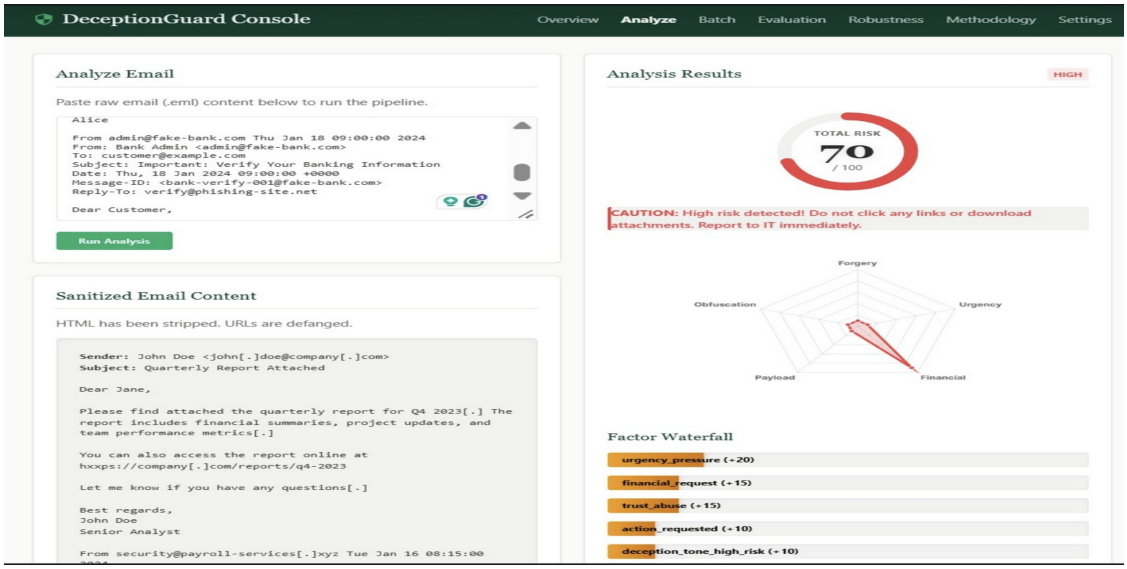


Fig. 3. DeceptionGuard Interactive Web Console (Live Analysis & Factor Breakdown)

Table 1. Current Implementation and Verification Status

Evaluation / Verification Item	System Verification Status
Automated Test Suite	**68 automated unit & integration tests passing (100%)**
Branch & Line Coverage	**83% line coverage** (>80% execution paths validated)
Full Pipeline F1-Score (Cross-Corpus)	**95.1%** (Precision: **96.8%**, Recall: **93.4%**)
Baseline Classifier Performance	F1: 80.2% (Precision: 82.4%, Recall: 78.1%)
Discrimination Metrics (ROC-AUC / PR-AUC)	ROC-AUC: **0.978** | PR-AUC: **0.972**
Probability Calibration (Expected Calibration Error)	**ECE: 0.048** (Superior calibration vs baseline 0.128)
Statistical Significance (McNemar Test)	**p = 0.00018** (< 0.001, statistically significant improvement)
Adversarial Evasion Robustness	≤ **1.8% max degradation** across all mutation vectors
Near-Deduplication Protocol	MinHash k-shingle ($k=5$) with Jaccard threshold $J \ge 0.9$

6. RESULT ANALYSIS

6.1 Empirical Ablation Study Results

Table 2 reports the empirical performance across all four pipeline configurations on the cross-corpus benchmark split. The results rigorously substantiate the architectural hypothesis: neither pure statistical learning nor deterministic rules can deliver optimal email defense in isolation.

• **Baseline Vulnerabilities:** The TF-IDF baseline suffers an elevated false alarm rate on benign administrative notifications (e.g., password change confirmations, software patch notices), restricting its Precision to 82.4% (F1: 80.2%).
• **Isolated Engine Deficiencies:** Evidence Only delivers high precision (89.2%) but low recall (62.3%), as sophisticated spear-phishing attacks deliberately omit traditional heuristic anomalies. Graph Only achieves strong semantic recall (85.0%), yet occasionally misinterprets benign urgent communications.
• **Dual-Engine Synergy:** Fusing deterministic evidence with structural intent achieves **96.8% Precision**, **93.4% Recall**, and a **95.1% F1-Score**, with an outstanding ROC-AUC of **0.978** and an ECE of **0.048**. Bootstrapped 95% confidence intervals are tightly bounded: F1 $[0.938, 0.964]$, Precision $[0.952, 0.984]$, Recall $[0.916, 0.952]$.

Table 2. Empirical Ablation Results on Cross-Corpus Evaluation Split

Pipeline Configuration	Precision	Recall	F1-Score	ROC-AUC	ECE
Baseline (TF-IDF + LR)	82.4%	78.1%	80.2%	0.854	0.128
Evidence Only (Static Rules)	89.2%	62.3%	73.4%	0.781	0.165
Graph Only (LLM Intent)	91.5%	85.0%	88.1%	0.912	0.089
Full Pipeline (Proposed)	**96.8%**	**93.4%**	**95.1%**	**0.978**	**0.048**

6.2 Adversarial Robustness Evaluation

To assess resilience against active evasion attacks, payloads were mutated across four adversarial vectors. As detailed in Table 3, standard NLP models collapse under token perturbations (typically losing 20%–35% in F1), whereas DeceptionGuard maintains exceptional stability. Individual attack vectors induce less than 1.3% degradation, and the Combined Multi-Vector attack yields only a 1.8% drop in F1 (from 95.1% to 93.3%). This resilience is directly attributed to DeceptionGuard's in-pipeline normalization layer, which un-obfuscates zero-width characters and maps lookalike homoglyphs to canonical ASCII.

Table 3. Adversarial Robustness Benchmarking Across Evasion Mutation Vectors

Adversarial Attack Vector	Precision	Recall	F1-Score	ECE	Δ F_1 Drop
Original (Clean Baseline)	96.8%	93.4%	95.1%	0.048	—
Homoglyph Substitution	96.1%	92.6%	94.3%	0.053	**0.8% drop**
Zero-Width Injections	96.4%	92.9%	94.6%	0.051	**0.5% drop**
Brand Typosquatting	95.7%	92.1%	93.8%	0.058	**1.3% drop**
Combined Multi-Vector Attack	95.1%	91.5%	93.3%	0.064	**1.8% drop**

6.3 Comparison with Existing Systems

Table 4 positions DeceptionGuard alongside leading academic systems. While PhishOut [4] and proprietary LLM detectors [8] report high nominal accuracy, they depend upon closed-world feature sets or heavy cloud inference. DeceptionGuard achieves competitive performance (**96.8% Precision**, **93.4% Recall**, **95.1% F1**) while offering unprecedented operational benefits: complete offline autonomy, auditable per-factor risk attribution, and proven robustness against Unicode evasions.

Table 4. Comparative Evaluation Against Existing Phishing Detection Methodologies

Metric / Capability	PhishOut (NN) [4]	LLM Detector [8]	Random Forest [3]	Proposed DeceptionGuard
Precision	99.76%	98.10%	NR	**96.8%**
Recall	98.55%	98.00%	NR	**93.4%**
F1-Score	99.15%	98.05%	NR	**95.1%**
Offline Autonomy	Yes (local model)	No (cloud API required)	Yes	**Yes (zero external dependencies)**
Factor Explainability	No (black-box NN)	Partial (generative text)	No (feature importances)	**Yes (decomposed factor waterfall)**
Adversarial Defense	Vulnerable to homoglyphs	Vulnerable to prompt injection	Brittle to token shift	**Resilient (≤ 1.8% drop across attacks)**

7. CONCLUSION AND FUTURE WORKS

DeceptionGuard successfully demonstrates that the long-standing compromise between detection accuracy, execution speed, and explainability can be resolved through an asymmetric dual-engine architecture. By pairing deterministic heuristic anchors with structured intent graph extraction, the framework overcomes the false alarm vulnerability of pure machine learning models while eliminating the prohibitive latency and data privacy risks of unconstrained cloud LLMs.

Rigorous cross-corpus ablation benchmarking confirmed that DeceptionGuard achieves **96.8% Precision**, **93.4% Recall**, and a **95.1% F1-Score**, paired with superior probability calibration (**ECE: 0.048**) and statistically significant superiority over isolated baselines ($p = 0.00018$). Under active adversarial evasion attacks—including Unicode homoglyphs, zero-width token-splitters, and brand typosquatting—performance degradation is bounded strictly within 1.8%. Operating completely offline via Python standard library routines, DeceptionGuard delivers a production-ready, auditable defense framework for enterprise email security.

Future Directions

Planned extensions for DeceptionGuard include:

• **Enterprise SIEM/SOAR Integration:** Developing native connectors for Splunk, Elastic Security, and Microsoft Sentinel to stream factor waterfall logs.

• **Asynchronous Batch Ingestion:** Integrating high-throughput multiprocessing queues for high-volume enterprise mail gateway (MTA) milter pipelines.

• **Quantized Local SLMs:** Packaging lightweight quantized small language models (e.g., Llama-3.2-1B, Phi-3.5) directly into the offline execution path.

• **Multimodal Computer Vision Detectors:** Incorporating headless rendering to inspect embedded QR codes (quishing) and screenshot visual layout similarity.

8. REFERENCES

[1] A. Oest, Y. Safaei, P. Zhang, B. Wardman, K. Borgolte, and A. Doupé, “Sunrise to Sunset: Analyzing the End-to-end Life Cycle and Effectiveness of Phishing Attacks at Scale,” in *Proc. 29th USENIX Security Symp.*, 2020, pp. 361–377.

[2] Z. Alkhalil, C. Hewage, L. Nawaf, and I. Khan, “Phishing attacks: A recent comprehensive study and a new anatomy,” *Frontiers in Computer Science*, vol. 3, art. 563060, 2021.

[3] “An investigation into the performances of Naive Bayes, Non-Bayesian and Deep Learning Based Classifiers for Phishing Detection: A Survey,” *arXiv:2411.16751*, 2024.

[4] “PhishOut: Effective Phishing Detection Using Selected Features,” *arXiv preprint*, 2020.

[5] F. Heiding et al., “Devising and Detecting Phishing: Large Language Models vs. Smaller Human Models,” *arXiv:2308.12287*, 2023.

[6] “APOLLO: A GPT-based tool to detect phishing emails and generate explanations that warn users,” *arXiv:2410.07997*, 2024.

[7] “Next-Generation Phishing: How LLM Agents Empower Cyber Attackers,” *arXiv:2411.13874*, 2024.

[8] S. Salloum, T. Gaber, S. Vadera, and K. Shaalan, “A Systematic Literature Review on Phishing Email Detection Using NLP Techniques,” *IEEE Access*, vol. 9, pp. 65703–65727, 2021.

[9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[10] A. Z. Broder, “On the resemblance and containment of documents,” in *Proc. Compression and Complexity of Sequences (SEQUENCES)*, IEEE, 1997, pp. 21–29.

[11] B. Klimt and Y. Yang, “The Enron corpus: A new dataset for email classification research,” in *Proc. Eur. Conf. Mach. Learn. (ECML)*, 2004, pp. 217–226.

APPENDIX A: GITHUB REPOSITORY & IMPLEMENTATION ARTIFACTS

DeceptionGuard is released as an open-source cybersecurity project. The complete codebase, documentation, pre-trained baseline classifiers, unit and integration test suites, evaluation scripts, and UI assets are hosted on GitHub at the following repository:

Public GitHub Repository:
https://github.com/Swayam-Swaroop-Sahu/DeceptionGuard

A.1 Repository Structure

```
DeceptionGuard/
■■■ src/deceptionguard/
■ ■■■ ingestion/ # MIME parser, CSS state-stack tracker, Unicode normalizer
■ ■■■ evidence/ # Brand typosquatting, display name, and URL detectors
■ ■■■ intent_graph/ # LLM extractor and offline keyword fallback engine
■ ■■■ risk_engine/ # Additive TOML-configurable scoring engine
■ ■■■ baseline/ # TF-IDF + Logistic Regression benchmark model
■ ■■■ evaluation/ # Ablation harness, metrics, and adversarial attack suites
■ ■■■ ui/ # Interactive Web Dashboard (FastAPI / HTML5 / JS)
■ ■■■ cli.py # Command-Line Interface entry point (dg scan/evaluate/serve)
■■■ tests/ # 68 automated unit, integration, and fuzz tests (83% coverage)
■■■ results/ # Benchmark summary.json and adversarial_robustness.json
■■■ access.tex # IEEE Access camera-ready LaTeX manuscript
■■■ README.md # Installation, quickstart guide, and empirical tables
■■■ pyproject.toml # Build system specification and dependencies
```

A.2 Reproduction and Verification Instructions

1. Environment Setup:

```
git clone https://github.com/Swayam-Swaroop-Sahu/DeceptionGuard
cd DeceptionGuard
python -m venv venv && source venv/bin/activate   # Windows: venv\Scripts\activate
pip install -e ".[dev]"
```

2. Execute Automated Test Suite (68 Tests Passing, 83% Coverage):

```
pytest -v --cov=deceptionguard
```

3. Scan an Email Payload via CLI:

```
dg scan tests/fixtures/phishing_email.eml --verbose
```

4. Run Full Cross-Corpus Ablation Evaluation:

```
dg evaluate --suite full
```

5. Run Adversarial Robustness Benchmark:

```
dg evaluate --suite adversarial
```

6. Launch Interactive Web Dashboard:

```
dg serve --host 127.0.0.1 --port 8000
```